

2.3. Retrieval-Augmented Generation (RAG)

Dentro del campo del Procesamiento de Lenguaje Natural encontramos, entre otras técnicas, RAG, que es una técnica avanzada en este contexto. Se basa en combinar la información de fuentes de datos con la generación de texto. Combina un modelo de recuperación diseñado para buscar grandes sets de datos con un modelo de lenguaje, que toma esa información y genera una respuesta.

La generación aumentada de recuperación puede mejorar la relevancia de una experiencia de búsqueda, al agregar contexto de fuentes de datos adicionales y complementando la base original del entrenamiento de un *large language model* (LLM), lo que mejora el resultado del modelo de lenguaje sin tener que volver a entrenar el modelo.

RAG es una técnica interesante para tareas como la respuesta a preguntas y la generación de contenido porque permite que los sistemas de inteligencia artificial generativa utilicen fuentes de información externas para producir respuestas más precisas y fieles al contexto. [16]

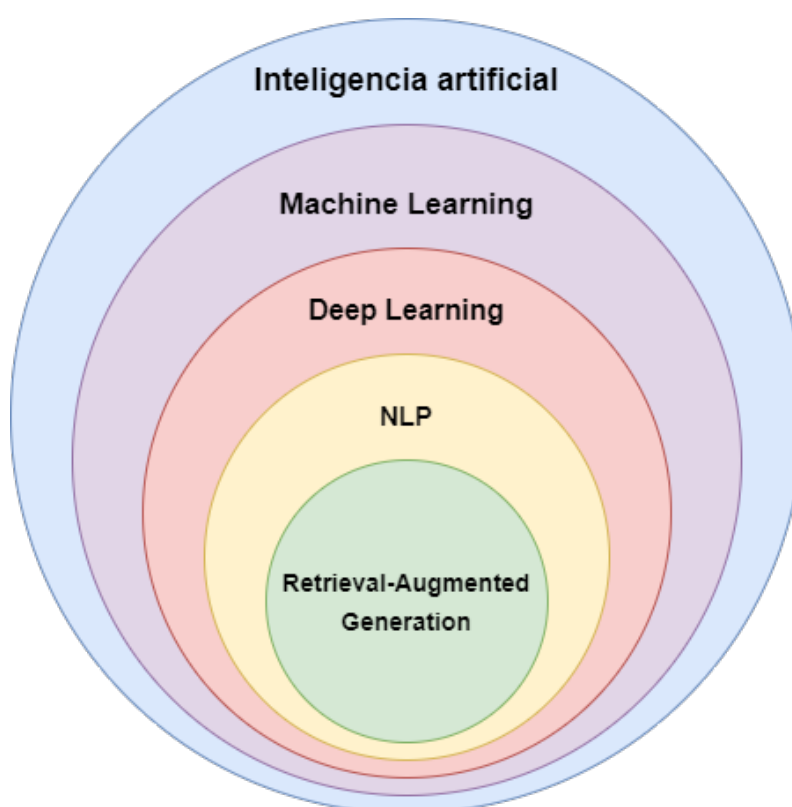


Figura 2.4: Jerarquía de los distintos campos expuestos

2.3.1. Funcionamiento

Retrieval-Augmented Generation utiliza modelos de lenguaje avanzados y técnicas de IA para llevar a cabo el proceso mencionado, que se divide en las siguientes etapas:

Recuperación de información: esta etapa comienza con la consulta por parte del usuario sobre algún aspecto determinado de la base de datos que se desea analizar. A continuación, el modelo realiza una búsqueda en una base de datos indexada o en fuentes externas y recupera la información que puede ser relevante para proporcionar una respuesta al usuario.

La recuperación dependerá de la consulta de entrada, que condicionará la búsqueda y dónde se realiza esta. Para esto se suelen utilizar técnicas que permiten realizar búsquedas de manera eficiente como son la búsqueda semántica o las coincidencias de palabras clave. La información recuperada se convierten en vectores que se almacenan en una base de datos vectorial y que serán ordenados según su relevancia para la búsqueda de entrada. [16]

El modelo de recuperación clasificará la información recuperada según su relevancia para la búsqueda de entrada, analizando las similitudes entre la consulta realizada y la información que hay en cada vector. Los documentos con las puntuaciones más altas se seleccionarán para su posterior procesamiento.

Generación de la respuesta: la información que ha sido recuperada de la base de datos se procesará de forma que ayude a la hora de generar una respuesta, la cual será creada por un modelo de generación basado en técnicas de *deep learning*, como es un LLM, que utiliza la información recuperada para generar las respuestas de texto. [16]

Las respuestas suelen ser más precisas contextualmente, ya que han sido moldeadas por la información de recuperación. Esto resulta especialmente importante en contextos donde los datos que hay en internet son escasos.

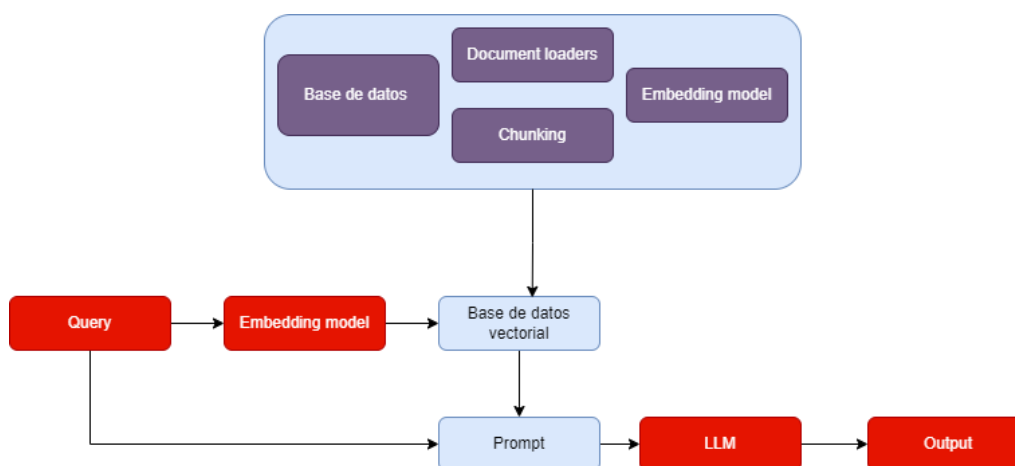


Figura 2.5: Diagrama funcionamiento RAG básico

En la figura 2.5 se presenta un esquema en el que se puede ver el flujo que sigue un sistema RAG. En primer lugar encontramos el bloque de procesamiento de documentos, que cuenta con la base de datos en la que se almacenan los documentos indexados para proporcionar información y contexto al modelo.

Además, este bloque también cuenta con *document loaders*, con los cuales se lleva a cabo el proceso de carga inicial de los documentos que se tienen como fuente de información de manera que estén en un formato adecuado y etiquetados de forma correcta.

Por otro lado, se realiza un procesamiento de esos datos recogidos en la etapa de *chunking*. En esta se realiza el proceso de división de los grandes documentos en partes más pequeñas (que reciben el nombre de *chunks*) de forma que se puedan manejar de una mejor manera.

En este bloque, además de los módulos ya mencionados, se encuentran los modelos de *embedding*. Su función es transformar el texto que hay en los *chunks* en representaciones numéricas de cada uno de ellos, lo que facilita el procesamiento de estos.

Por último, una vez se han obtenidos los vectores numéricos que representan a todos los *chunks* en los que se ha dividido la base de datos original. Estos vectores se almacenan en una base de datos vectorial, que permitirá recuperar la información de una forma más rápida cuando se realice una búsqueda de similitudes en el contenido. [16]

Por otra parte, encontramos el bloque *query*, que se refiere a las consultas que realiza el usuario relacionadas con la base de datos indexada, esta será procesada por otro modelo de *embedding* de manera que quede convertida también en un vector numérico.

Para encontrar las similitudes que hay entre lo que el usuario ha dado como entrada y lo que hay en la base de datos del sistema se realizará una búsqueda semántica en la base de datos vectorial, con lo que se obtendrán los *chunks* más relevantes.

A continuación encontramos el *prompt*, que consiste en un conjunto de instrucciones a seguir, que se pasarán, junto con la *query* del usuario y con los *chunks* recuperados, como entrada a un *large language model* (LLM). [16]

Finalmente encontramos el LLM, que tendrá la misión de generar, ciñéndose a las instrucciones recibidas, una respuesta adecuada a la consulta del usuario.

En la figura 2.6 se presenta un esquema que describe el proceso que ocurre en el bloque de procesamiento de los documentos, comentado anteriormente:

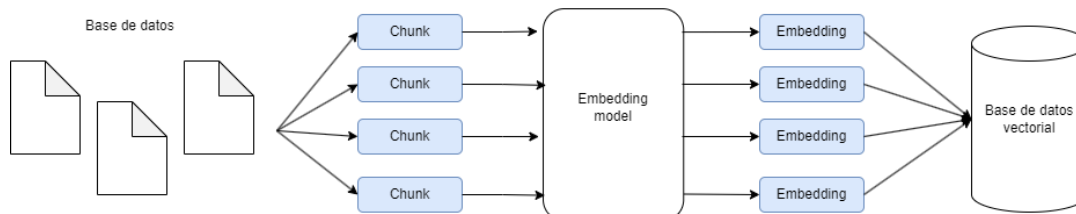


Figura 2.6: Esquema carga en base de datos vectorial

2.3.2. Aplicaciones

Existen varios campos en los que podemos ver la utilidad de *Retrieval-Augmented Generation* [16], como pueden ser:

Sistemas Question-Answer: RAG es capaz de mejorar la calidad de las respuestas en este tipo de sistemas, sobre todo en ámbitos en los que es necesario que se responda con mucha precisión.

Asistentes Virtuales: esta técnica facilita la generación de respuestas en aplicaciones como chatbots, ayudando a que estas sean más informativas y claras.

Síntesis de información: *Retrieval-Augmented Generation* puede ser utilizado para unir información proveniente de diferentes fuentes o de bases de datos muy grandes de manera que sea más fácil que el usuario pueda acceder a información más específica.

2.3.3. Beneficios

Esta técnica tiene varios beneficios sobre los modelos de lenguaje que funcionan de forma aislada. En primer lugar, podemos apreciar que RAG permite que los modelos sean capaces de acceder a fuentes de datos externas más recientes y actualizadas de forma eficiente, ya que puede actualizar sus referencias más rápidamente.

Esto se debe a que al aportar información renovada a un modelo de RAG, esto no afectará al modelo de una forma global, solamente provocará que la base de datos indexada que había anteriormente quede sustituida o ampliada, algo que solo modificará el funcionamiento de la fase de recuperación de información, que tendrá un conjunto de datos a analizar más grande o distinto. Por tanto, no será necesario hacer un reentrenamiento del modelo para mejorar la precisión o la relevancia en las repuestas del modelo, algo que por otra parte, hará que se asuma un menor coste computacional.

Por otra parte, cuando un LLM se enfrenta a una búsqueda compleja, en ocasiones se puede producir 'alucinaciones', que son situaciones en las que un modelo genera información falsa o imprecisa en relación con la entrada proporcionada a pesar de parecer correctas.

La generación aumentada por recuperación consigue disminuir este problema ya que, al contar con un paso previo de recuperación de la información antes de proporcionar una respuesta al usuario, el modelo no depende solamente de lo que haya en la base de datos que se ha utilizado para entrenarlo, sino que también cuenta con los documentos que le han sido indexados de forma externa, ofreciendo una información añadida que puede ser de ayuda para generar una respuesta más completa y precisa.